

INTELLIQUERY: AI-POWERED CONVERSATIONAL BUSINESS INTELLIGENCE DASHBOARD PLATFORM

Functional Design

by

Izma Khan 26926

Sameed Ahmad 26956

Shaikh Muhammad Umair 26409

Supervised by

Abeera Tariq

Lecturer and Academic Director

Department of Computer Science

Institute of Business Administration (IBA), Karachi

Industry Supervisor:

Muhammad Naeem

Folio3 Software Solutions

Fall Semester 2025

Institute of Business Administration (IBA) Karachi Pakistan

Contents

List of Tables	iv
List of Figures	v
1 Introduction	1
1.1 Project Overview	1
1.2 Implementation Phases	1
1.3 Technology Stack (As Implemented)	2
2 Interaction Diagrams	3
2.1 Natural Language to SQL Query Processing	4
2.2 Error Recovery and Retry Flow	6
2.3 Context-Aware Follow-up Query	8
2.4 LangGraph Workflow Execution	9
3 Class Diagram and Interface Specification	10
3.1 Overview Class Diagram	10
3.2 Detailed State Management Classes	11
3.3 Node Classes - Processing Pipeline	12
3.4 Tool Classes and Error Management	13
3.5 Database Context Classes	13
4 System Architecture and System Design	14
4.1 Architectural Style	14
4.2 System Architecture Diagram	15
4.3 Identifying Subsystems	15
4.3.1 Subsystem Descriptions	15
5 Mapping Subsystems to Hardware	17
5.1 Deployment Architecture	17
5.2 Hardware Mapping Table	18
5.3 Network Communication	18

6	Persistent Data Storage	19
6.1	PostgreSQL Database Schema (Northwind)	19
6.1.1	Key Tables Description	20
6.2	AgentState Persistence (MemorySaver)	20
6.3	ChromaDB Vector Store	20
7	Network Protocol	21
7.1	Communication Protocols	21
8	Global Control Flow	22
8.1	Execution Model	22
8.2	Concurrency Handling	22
8.3	Time Dependencies	23
9	Hardware and Software Requirements	24
9.1	Client Requirements (Planned)	24
9.2	Server Requirements (Current)	24
9.3	Cloud Services	25
10	Algorithms and Data Structures	26
10.1	Key Algorithms	26
10.1.1	Few-Shot Example Selection Algorithm	26
10.1.2	SQL Validation Algorithm	26
10.1.3	SQL Query Cleaning Algorithm	27
10.2	Data Structures	28
10.2.1	Hash Maps / Dictionaries	28
10.2.2	Lists/Arrays	28
10.2.3	Dataclasses	29
10.2.4	Vector Embeddings (NumPy Arrays)	29
11	User Interface Design and Implementation	30
11.1	Initial Design from SRS	30
11.2	Design Modifications for Implementation	30
11.2.1	Modified Design Decisions	31
11.3	Web Interface Implementation (Phase 2)	31
11.3.1	Planned Components	31
11.3.2	Technical Implementation Stack	33
12	Design of Tests	34
12.1	Unit Tests	34
12.1.1	Example Test Case	34

12.2	Integration Tests	35
12.2.1	End-to-End Test Example	35
12.3	Test Coverage	36
13	Conclusion	37

List of Tables

1.1	Current technology stack	2
5.1	Subsystem to hardware mapping	18
6.1	Northwind dataset tables	20
11.1	UI design modifications from SRS to FD	31
11.2	Web frontend technology stack	33
12.1	Unit test cases	34
12.2	Integration test cases	35

List of Figures

2.1	Complete flow from user query to SQL execution	4
2.2	Error handling with retry logic and error manager	6
2.3	Processing follow-up queries using conversation context	8
2.4	Complete LangGraph StateGraph with conditional edges	9
3.1	High-level class relationships showing all major components	10
3.2	State management subsystem with full attributes and methods	11
3.3	Node classes responsible for query processing stages	12
3.4	Utility tools and error management classes	13
3.5	Database schema and context management	13
4.1	Complete system architecture showing all components and data flow	15
4.2	UML package diagram showing subsystem organization	15
5.1	Physical deployment showing subsystem-to-hardware mapping	17
6.1	Northwind database Entity-Relationship diagram	19
11.1	Business User Dashboard from SRS - Split-screen layout with chat and visual- izations	32
11.2	Conversation History page from SRS - Card-based layout	33

Abstract

This Functional Design document presents the comprehensive system design for IntelliQuery, an AI-powered conversational business intelligence platform. The document details interaction diagrams, class structures, system architecture, and implementation specifications for the system as currently implemented.

IntelliQuery addresses the challenge of democratizing business intelligence by enabling non-technical users to interact with complex data through natural language queries. The current implementation uses PostgreSQL database with Northwind dataset, employing a LangGraph-based agentic architecture with Google Gemini AI for query generation. Future phases will integrate web-based visualization dashboards and Power BI connectivity.

Implementation Status: Phase 1 (SQL Agent Backend) - Implemented. Phase 2 (Web Dashboard & Visualization) - In Progress. Phase 3 (Power BI Integration) - Planned.

Keywords: Conversational AI, Business Intelligence, LangGraph, Natural Language Processing, SQL Query Generation, PostgreSQL, Google Gemini, Agentic AI, Northwind Dataset

Chapter 1

Introduction

1.1 Project Overview

IntelliQuery is an AI-powered conversational business intelligence platform designed to bridge the gap between complex data analytics and non-technical business users. This Functional Design document translates requirements into concrete system specifications, architectural decisions, and implementation details reflecting the actual current implementation.

The system enables users to query business data using natural language, automatically generating appropriate SQL queries through an intelligent agent-based workflow coordinated by LangGraph. By eliminating the need for SQL expertise, IntelliQuery democratizes access to business intelligence across organizations.

1.2 Implementation Phases

Phase 1: SQL Agent Backend (Implemented)

- Natural language to SQL conversion using Google Gemini 2.0 Flash
- LangGraph agentic workflow with 5 specialized nodes
- PostgreSQL (AWS RDS) Northwind database integration
- Schema-aware query generation with few-shot learning
- Comprehensive error handling and retry logic with ErrorManager
- Context management for follow-up queries
- Semantic similarity-based example selection using ChromaDB

Phase 2: Web Dashboard (In Progress)

- FastAPI RESTful backend API

- React/Next.js web frontend
- Dynamic visualization generation (Plotly/Chart.js)
- User authentication and session management
- Conversation history persistence

Phase 3: Advanced Features (Planned)

- Power BI integration layer
- DAX query generation capabilities
- Advanced analytics and insights
- Export functionality (PDF, Excel, PowerPoint)

1.3 Technology Stack (As Implemented)

Component	Technology
AI/NLP Engine	Google Gemini 2.0 Flash
Workflow Orchestration	LangGraph (StateGraph)
Database	PostgreSQL on AWS RDS
Vector Store	ChromaDB
Embeddings	FastEmbed (sentence-transformers/all-MiniLM-L6-v2)
Backend Framework	Python 3.11, FastAPI (planned)
Message System	LangChain Core Messages
State Management	Dataclasses with LangGraph Checkpointer
Dataset	Northwind (Customers, Orders, Products, etc.)

Table 1.1: Current technology stack

Chapter 2

Interaction Diagrams

This chapter presents sequence diagrams for major use cases, showing how system components interact in the current implementation.

2.1 Natural Language to SQL Query Processing

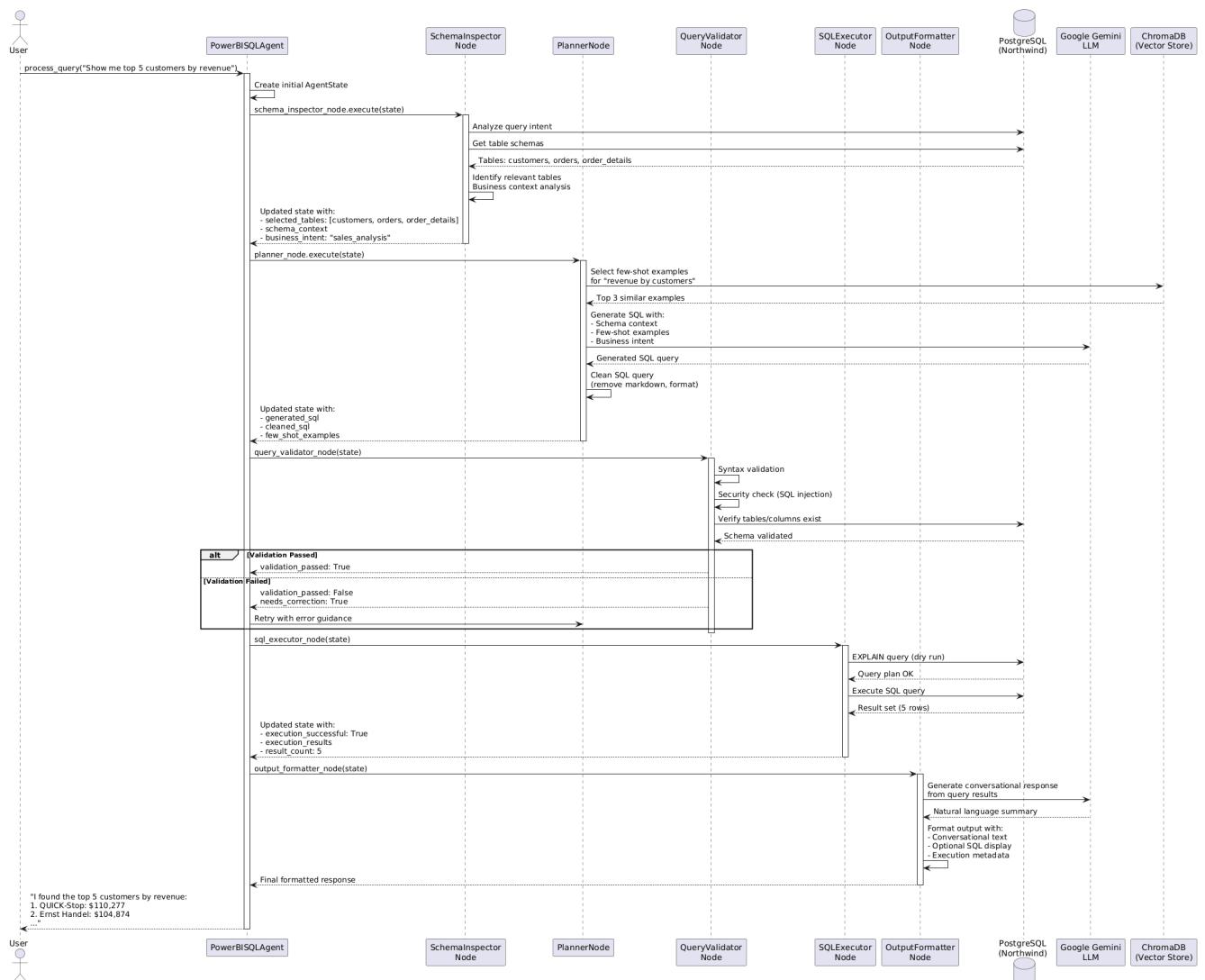


Figure 2.1: Complete flow from user query to SQL execution

Flow Description:

1. User submits natural language query (e.g., "Show me top 5 customers by revenue")
2. **PowerBISQLAgent** receives query and creates initial AgentState
3. **SchemaInspectorNode** analyzes query to identify relevant tables:
 - Analyzes query intent using schema context
 - Identifies relevant tables from Northwind database
 - Retrieves schema information for selected tables

- Returns updated state with `selected_tables`, `schema_context`, `business_intent`

4. **PlannerNode** generates SQL query:

- Queries ChromaDB for semantically similar few-shot examples
- Constructs prompt with schema context and examples
- Calls Google Gemini LLM to generate SQL
- Cleans generated SQL (removes markdown, formats code)
- Returns updated state with `generated_sql` and `cleaned_sql`

5. **QueryValidatorNode** validates generated SQL:

- Checks syntax correctness
- Validates security (SQL injection prevention)
- Verifies tables and columns exist in schema
- Checks business logic compliance
- If validation fails, marks `needs_correction` and provides error details

6. **Conditional Edge**: If validation passed, proceed to execution; if failed and retries ≥ 3 , return to **PlannerNode** with error guidance

7. **SQLExecutorNode** executes validated query:

- Runs EXPLAIN query for dry run validation
- Executes actual SQL query against PostgreSQL
- Captures execution time and result count
- Returns execution results

8. **OutputFormatterNode** formats response:

- Uses Google Gemini to generate conversational response
- Formats results in natural language
- Optionally includes SQL query and execution metadata
- Updates conversation context

9. System returns formatted response to user

2.2 Error Recovery and Retry Flow

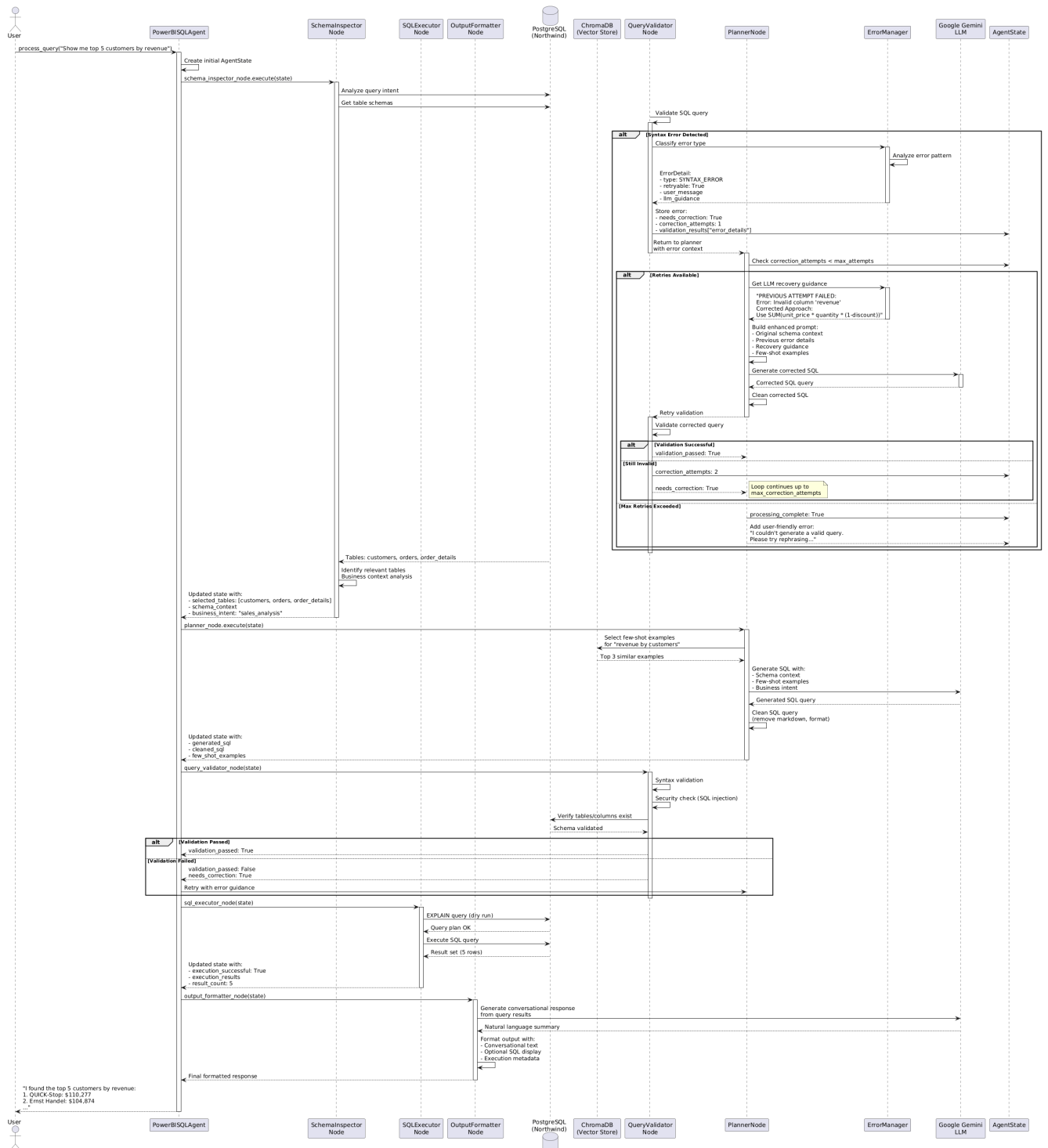


Figure 2.2: Error handling with retry logic and error manager

Flow Description:

The system implements sophisticated error recovery when query validation or execution fails:

1. QueryValidator detects error in generated SQL
2. ErrorManager classifies error type (syntax, schema, security, business logic)
3. System determines if error is retryable
4. Error details and recovery guidance are added to state
5. State is routed back to PlannerNode with error context
6. PlannerNode generates enhanced prompt including:
 - Previous error details
 - LLM-friendly recovery guidance from ErrorManager
 - Original schema context
7. Google Gemini generates corrected SQL query
8. Validation is re-attempted
9. Process repeats up to max_correction_attempts (default: 3)
10. If max retries exceeded, error is returned to user with helpful message

2.3 Context-Aware Follow-up Query

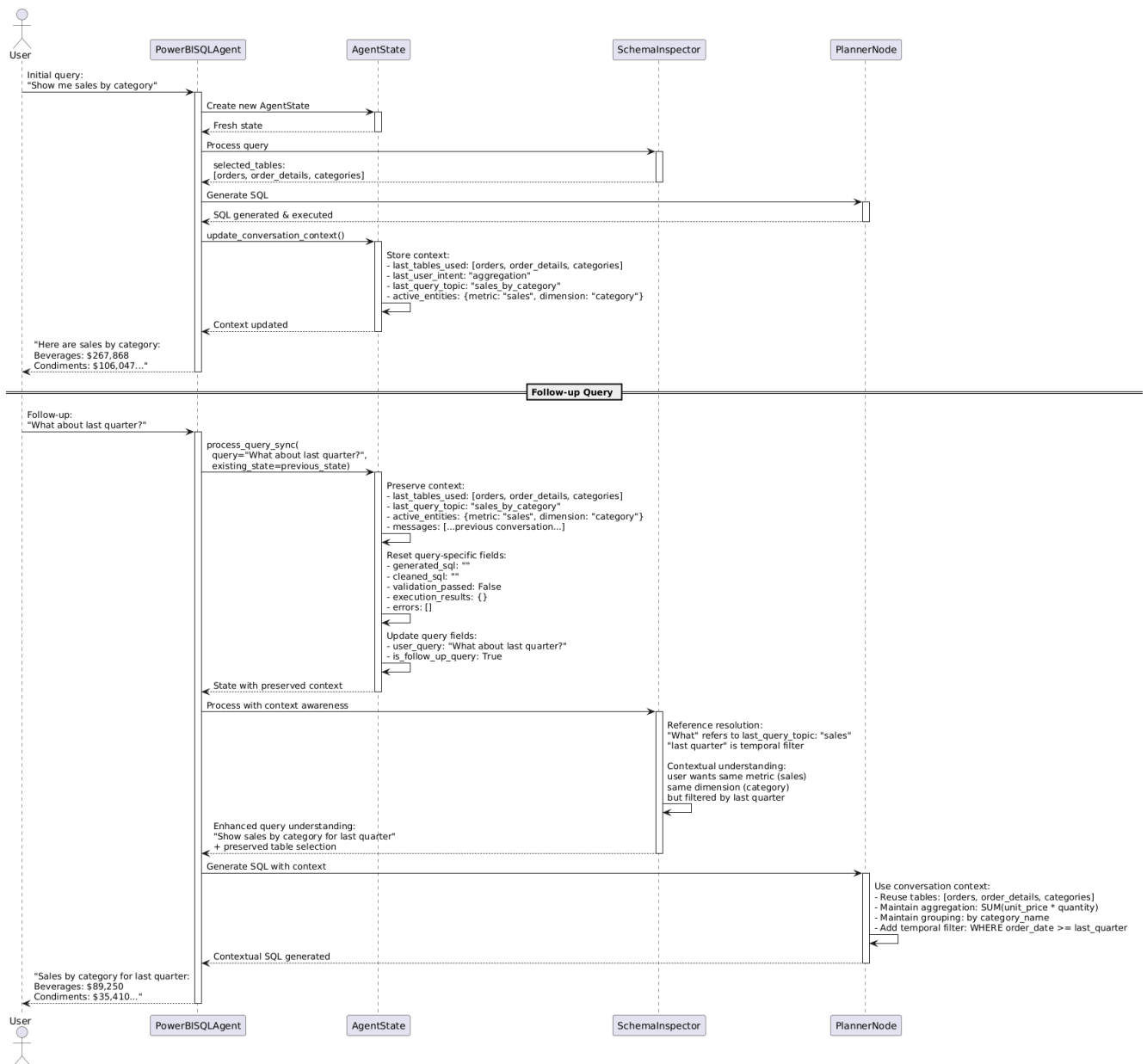


Figure 2.3: Processing follow-up queries using conversation context

Implementation Details:

The system maintains conversation context through AgentState fields:

```

1 # Context preservation in AgentState
2 conversation_context: Dict[str, Any]
3 last_query_topic: str = ""
4 last_tables_used: List[str] = []
5 last_result_summary: str = ""
6 last_user_intent: str = ""
  
```

```

7 active_entities: Dict[str, Any] = {}
8 context_window: List[Dict[str, Any]] = []
9
10 # Update context after successful query
11 def update_conversation_context(self, execution_results):
12     self.last_tables_used = self.selected_tables
13     self.last_user_intent = self.business_intent
14     self.last_query_topic = self._extract_topic()
15     # Store result summary for reference resolution

```

2.4 LangGraph Workflow Execution

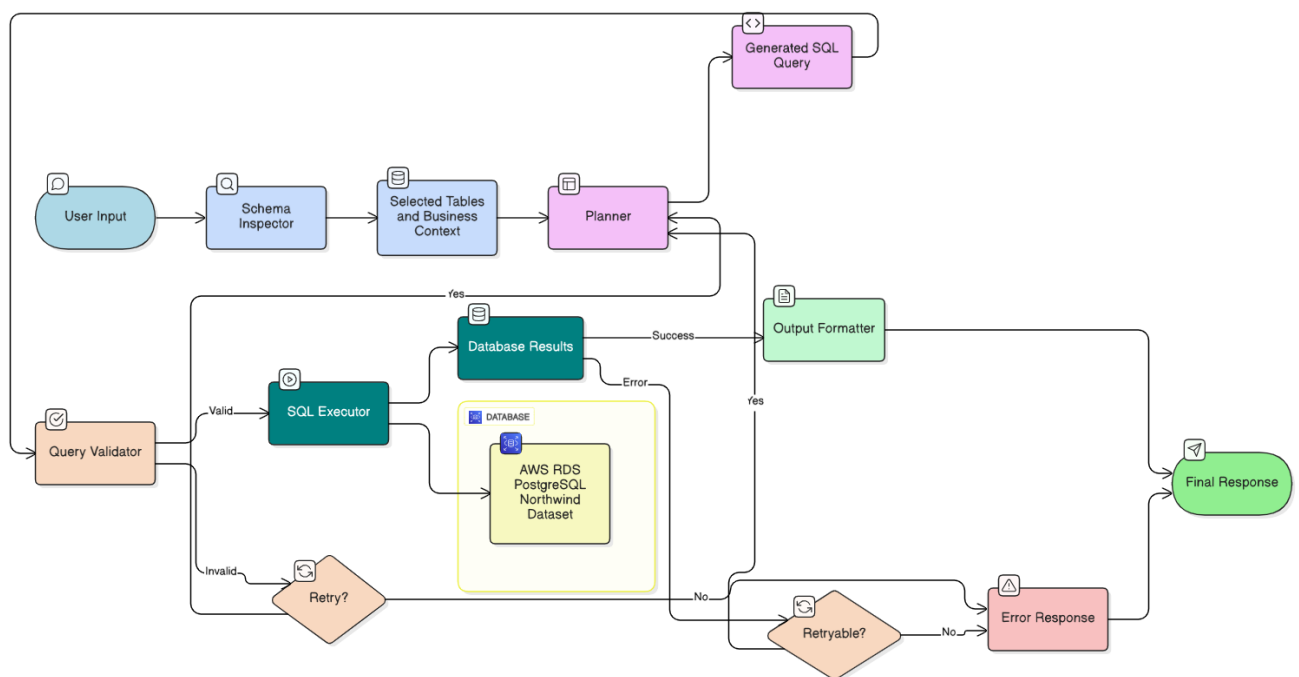


Figure 2.4: Complete LangGraph StateGraph with conditional edges

Workflow Architecture:

The system uses LangGraph's StateGraph for workflow orchestration with the following structure:

- **Nodes:** 5 specialized processing nodes
- **Edges:** Conditional routing based on state
- **Checkpoint:** MemorySaver for conversation persistence
- **State:** AgentState dataclass with comprehensive tracking

Chapter 3

Class Diagram and Interface Specification

This chapter presents the object-oriented design of IntelliQuery as actually implemented, showing classes, attributes, methods, and relationships from the codebase.

3.1 Overview Class Diagram

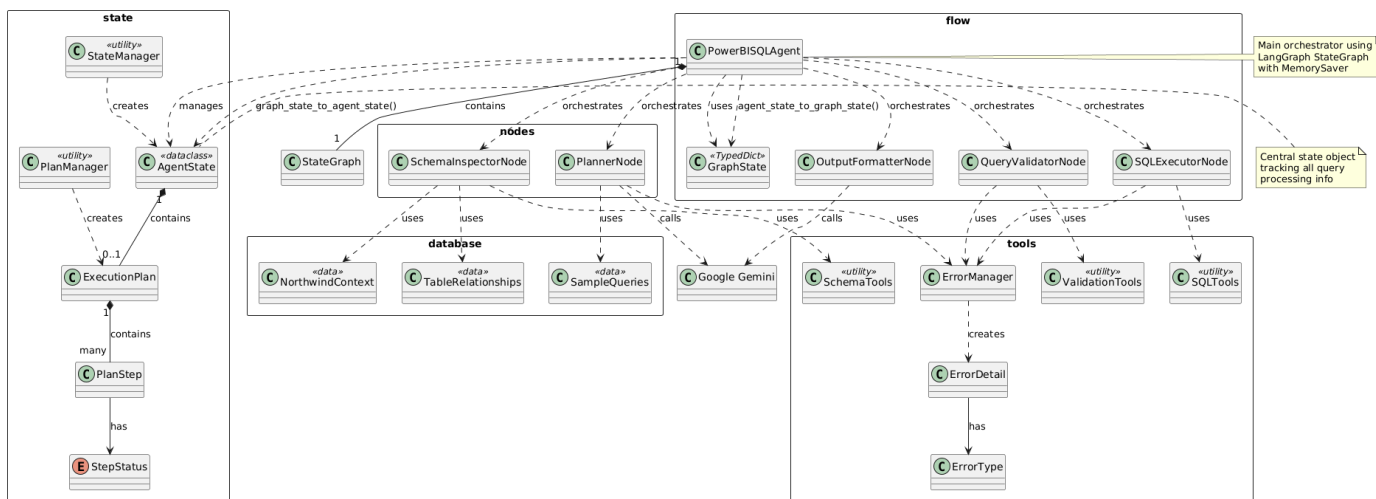


Figure 3.1: High-level class relationships showing all major components

3.2 Detailed State Management Classes

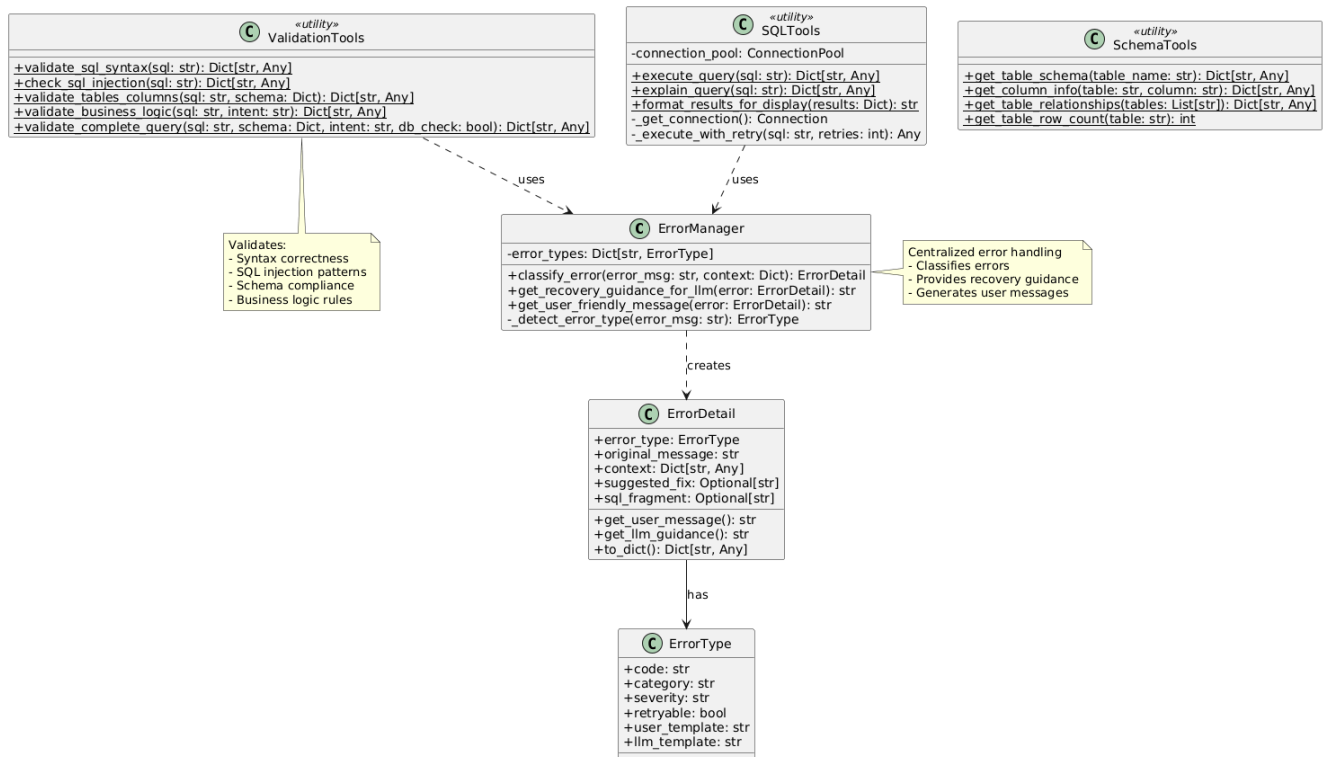


Figure 3.2: State management subsystem with full attributes and methods

3.3 Node Classes - Processing Pipeline

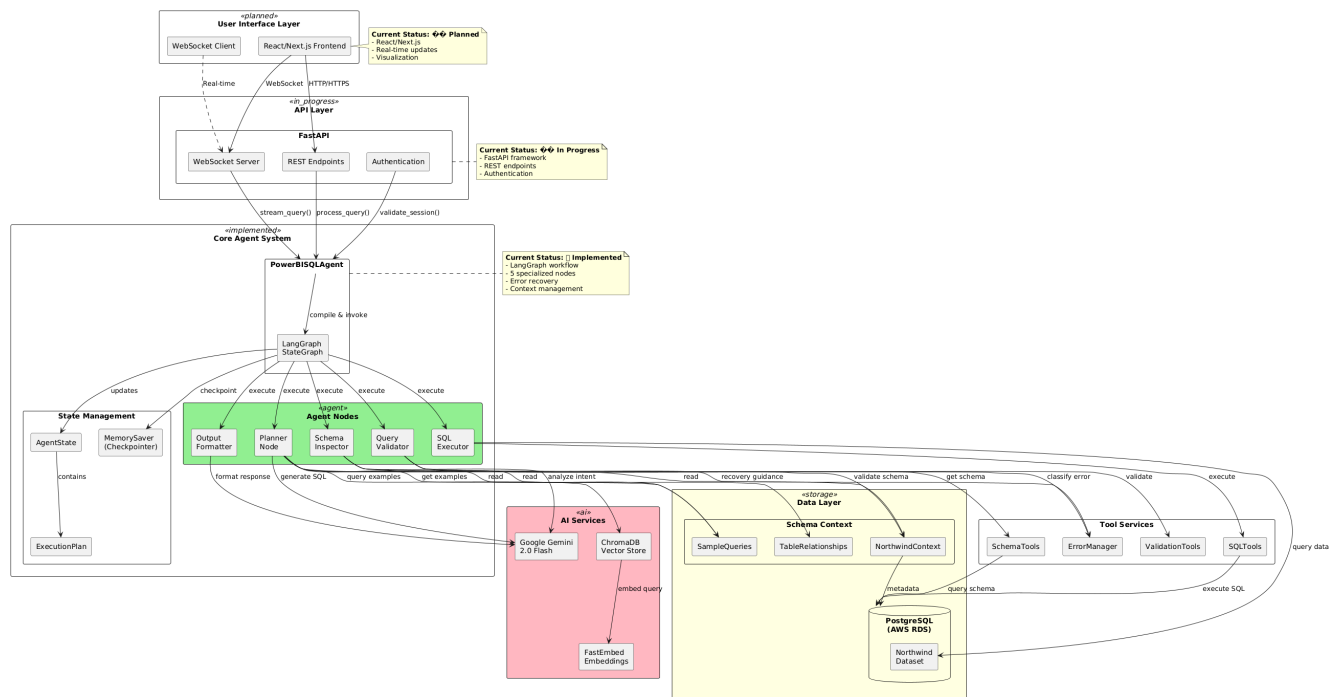


Figure 3.3: Node classes responsible for query processing stages

3.4 Tool Classes and Error Management

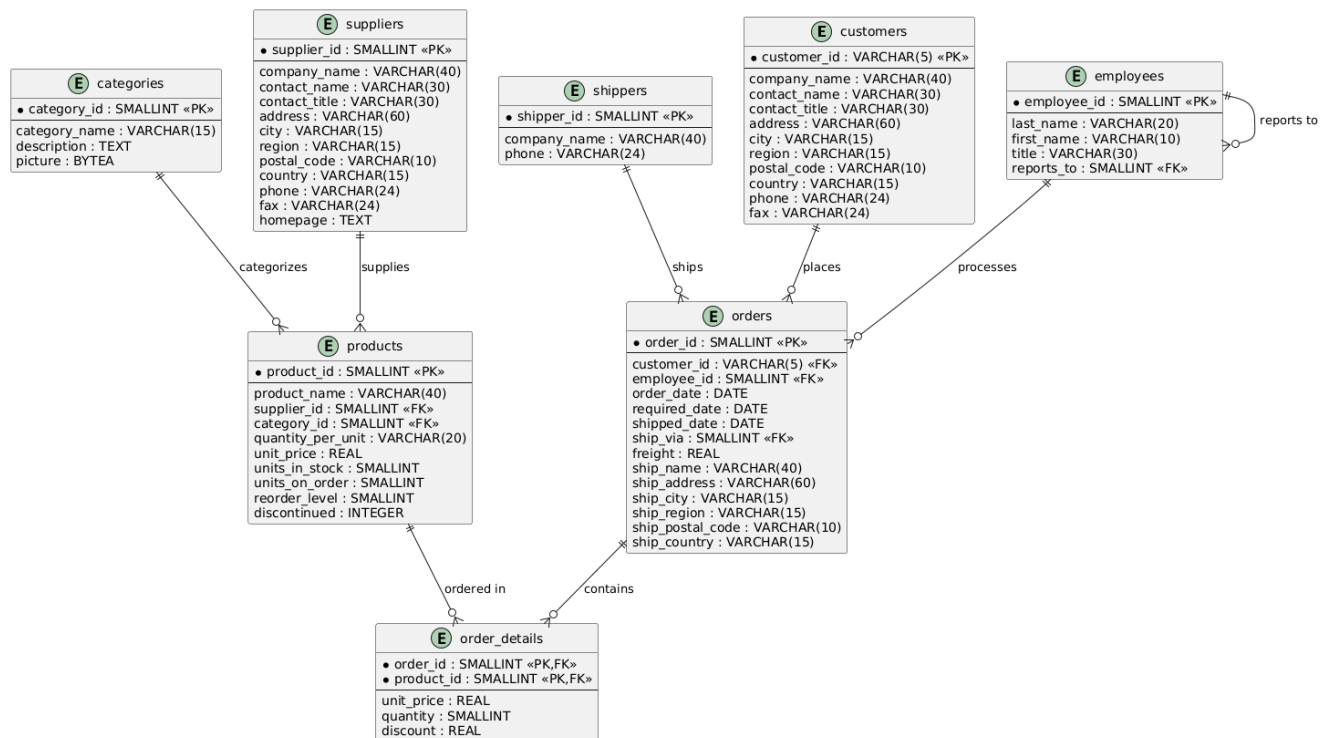


Figure 3.4: Utility tools and error management classes

3.5 Database Context Classes

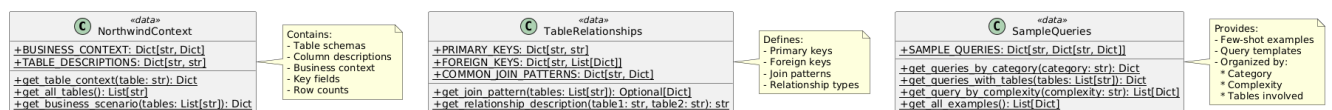


Figure 3.5: Database schema and context management

Chapter 4

System Architecture and System Design

4.1 Architectural Style

IntelliQuery employs an **Agent-Based Microservices Architecture** with **Event-Driven Pipeline** orchestrated by LangGraph. This architectural choice provides:

- **Modularity:** Each agent (node) is independent and focused on a single responsibility
- **Scalability:** Individual nodes can be optimized or scaled independently
- **Maintainability:** Clear separation of concerns makes debugging and updates easier
- **Resilience:** Error handling at each node with retry capabilities
- **Flexibility:** Easy to add new nodes or modify workflow logic
- **Traceability:** Complete state tracking through execution plan

Key Architectural Patterns:

1. **StateGraph Pattern:** LangGraph manages workflow as directed graph with conditional edges
2. **Agent Pattern:** Specialized nodes act as autonomous agents with specific capabilities
3. **Retry Pattern:** Automatic retry with error context for recoverable failures
4. **Context Preservation Pattern:** Conversation state persisted across queries
5. **Few-Shot Learning Pattern:** Dynamic example selection using semantic similarity

4.2 System Architecture Diagram

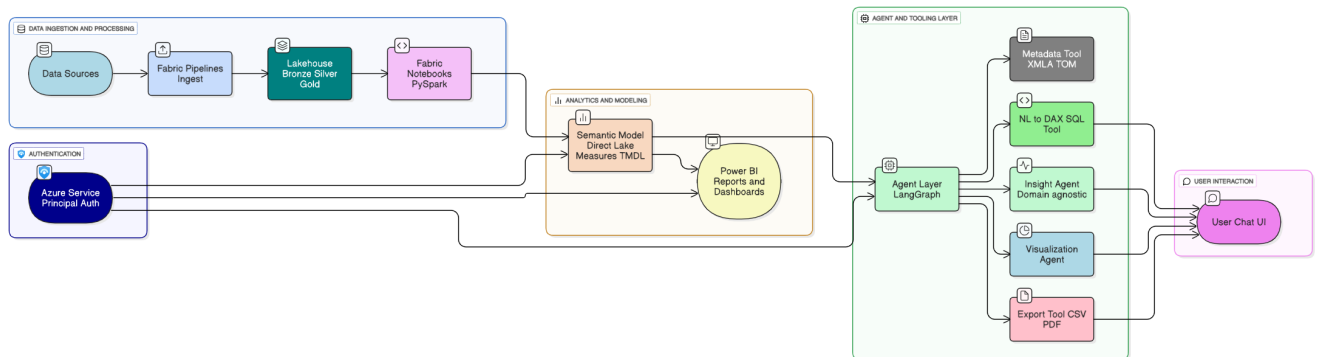


Figure 4.1: Complete system architecture showing all components and data flow

4.3 Identifying Subsystems

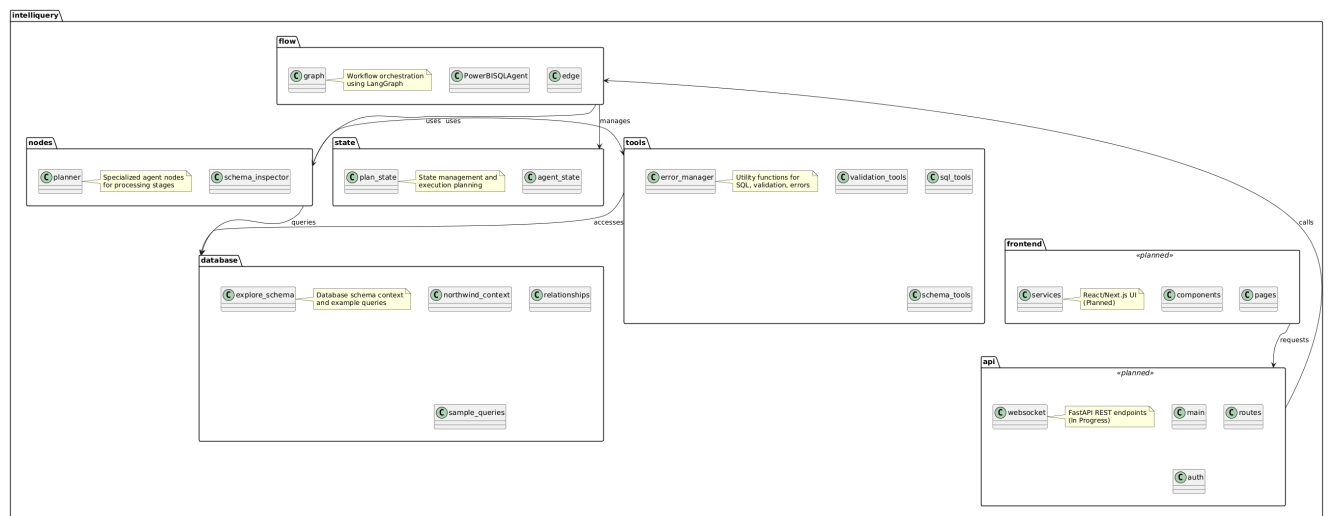


Figure 4.2: UML package diagram showing subsystem organization

4.3.1 Subsystem Descriptions

flow Package (Implemented):

- `graph.py`: LangGraph StateGraph definition and workflow compilation
- `edge.py`: Conditional edge functions for routing decisions
- Contains PowerBISQLAgent main orchestrator

nodes Package (Implemented):

- `schema_inspector.py`: Table identification and schema analysis
- `planner.py`: SQL generation with few-shot learning
- Each node is self-contained with `execute()` method

state Package (Implemented):

- `agent_state.py`: `AgentState` dataclass with all tracking fields
- `plan_state.py`: `ExecutionPlan` and `PlanStep` for workflow tracking
- `StateManager` and `PlanManager` utilities

tools Package (Implemented):

- `sql_tools.py`: Database connection and query execution
- `validation_tools.py`: SQL validation (syntax, security, schema)
- `schema_tools.py`: Schema metadata retrieval
- `error_manager.py`: Centralized error classification and recovery

database Package (Implemented):

- `northwind_context.py`: Business context and table descriptions
- `relationships.py`: Primary/foreign keys and join patterns
- `sample_queries.py`: Few-shot examples organized by category
- `explore_schema.py`: Dynamic schema exploration functions

api Package (In Progress):

- FastAPI application setup
- REST endpoints for query processing
- WebSocket for real-time streaming
- Authentication and session management

frontend Package (Planned):

- React components for chat interface
- Visualization components (Chart.js/Plotly)
- Service layer for API communication

Chapter 5

Mapping Subsystems to Hardware

5.1 Deployment Architecture

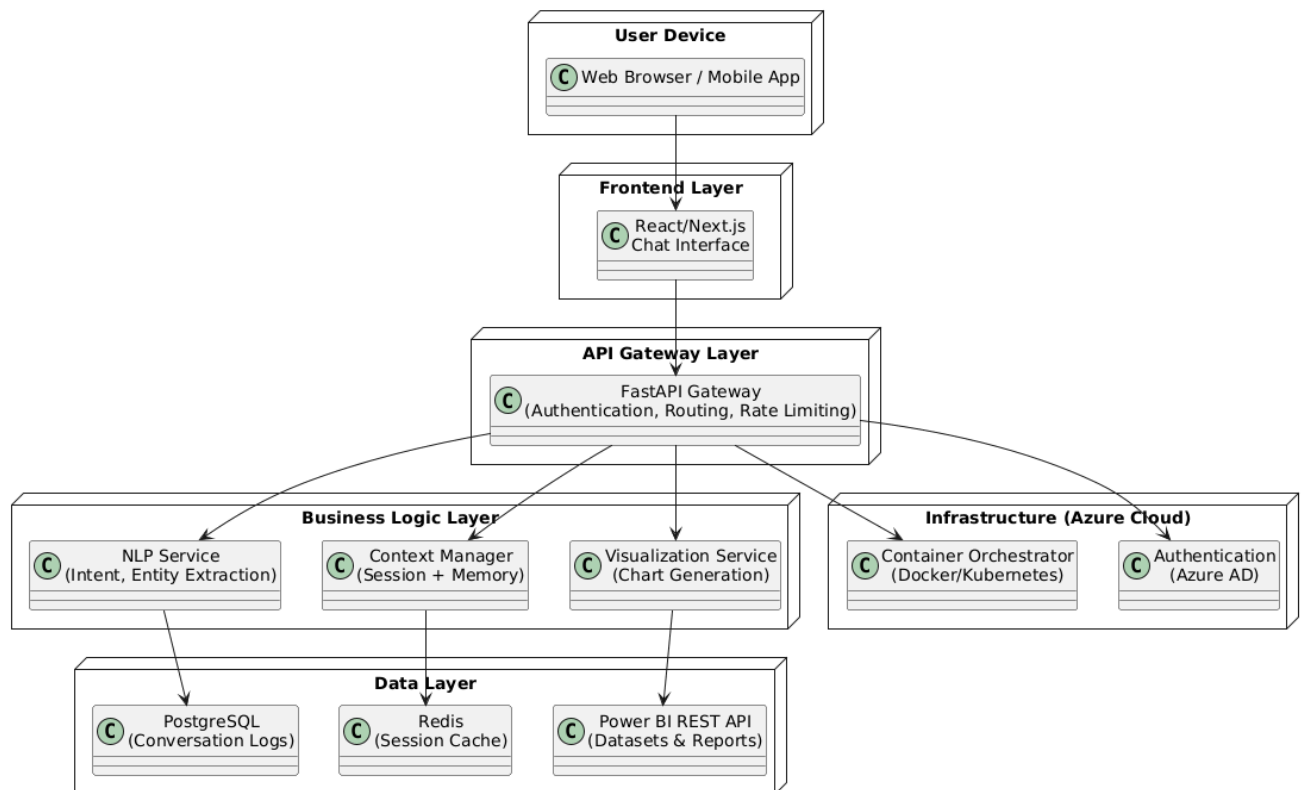


Figure 5.1: Physical deployment showing subsystem-to-hardware mapping

5.2 Hardware Mapping Table

Subsystem	Hardware/Platform	Status	Notes
React Frontend	Client Browser	Planned	Chrome, Firefox, Safari, Edge
FastAPI Backend	AWS EC2 / Local Dev	In Progress	Python 3.11, Uvicorn
Agent System	Application Server	Implemented	LangGraph workflow
Schema Inspector	Application Server	Implemented	CPU-bound processing
Planner Node	Application Server	Implemented	Calls external AI API
Query Validator	Application Server	Implemented	CPU-bound validation
SQL Executor	Application Server	Implemented	I/O-bound database calls
Output Formatter	Application Server	Implemented	Calls external AI API
Google Gemini API	Google Cloud	External	SaaS AI service
ChromaDB	Local/Docker Container	Implemented	Vector database
PostgreSQL	AWS RDS	Implemented	Managed database service
MemorySaver	Application Server	Implemented	In-memory/disk persistence

Table 5.1: Subsystem to hardware mapping

5.3 Network Communication

Internal Communication (Within Application Server):

- Function calls between Python modules (no network)
- Shared memory for state management

External Communication:

- **Client FastAPI:** HTTPS REST API / WebSocket WSS
- **FastAPI Google Gemini:** HTTPS REST API with API key authentication
- **Application PostgreSQL:** TCP/IP with psycopg2 driver
- **Application ChromaDB:** HTTP REST API (local) or gRPC

Chapter 6

Persistent Data Storage

6.1 PostgreSQL Database Schema (Northwind)

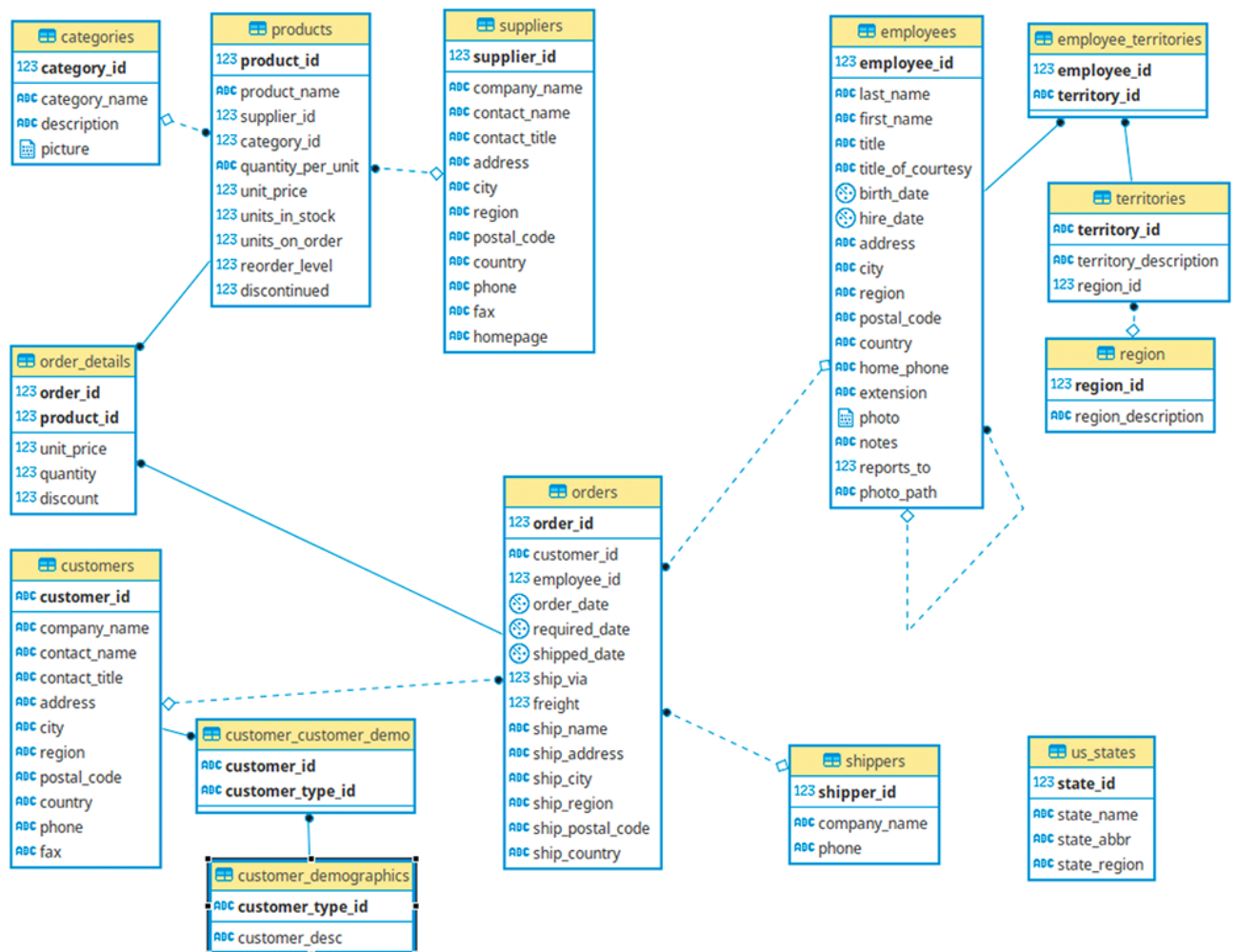


Figure 6.1: Northwind database Entity-Relationship diagram

6.1.1 Key Tables Description

Table	Rows	Purpose
customers	91	Customer information
orders	830	Order headers with dates, freight
order_details	2,155	Order line items (products, quantities, prices)
products	77	Product catalog
categories	8	Product categories
suppliers	29	Supplier information
employees	9	Employee records
shippers	3	Shipping companies

Table 6.1: Northwind dataset tables

6.2 AgentState Persistence (MemorySaver)

The LangGraph MemorySaver checkpoints AgentState for conversation continuity:

```
1 # Checkpoint configuration
2 config = {
3     "configurable": {
4         "thread_id": session_id # Unique per conversation
5     }
6 }
7
8 # State is automatically persisted at each node
9 self.graph = workflow.compile(checkpointer=self.memory)
```

Storage: In-memory (development) or disk-based (production)

6.3 ChromaDB Vector Store

Stores few-shot SQL examples with embeddings:

```
1 # Example document structure
2 {
3     "input": "Show me top 5 customers by revenue",
4     "query": "SELECT c.company_name, SUM(...) FROM ...",
5     "complexity": "medium",
6     "tables": "customers, orders, order_details"
7 }
```

Chapter 7

Network Protocol

7.1 Communication Protocols

1. Client to Backend (Planned):

- Protocol: HTTPS REST API
- Format: JSON
- Authentication: JWT tokens

2. Backend to Google Gemini:

- Protocol: HTTPS REST
- Authentication: API Key in headers
- Endpoint: `https://generativelanguage.googleapis.com/v1beta/...`

3. Backend to PostgreSQL:

- Protocol: TCP/IP
- Driver: psycopg2
- Port: 5432
- Connection pooling: Implemented in SQLTools

Chapter 8

Global Control Flow

8.1 Execution Model

LangGraph Event-Driven Pipeline:

- **Type:** Synchronous pipeline with conditional branching
- **Orchestrator:** LangGraph StateGraph
- **State Passing:** Each node receives and returns AgentState
- **Control Flow:** Conditional edges determine next node based on state

8.2 Concurrency Handling

Multi-User Support:

```
1 # Each user query runs in separate execution
2 async def process_query(user_query , session_id):
3     # Creates isolated state per session
4     initial_state = StateManager.create_initial_state(user_query)
5
6     # Independent workflow execution
7     final_state = await self.graph.ainvoke(initial_state , config)
```

Concurrent Execution:

- Python async/await for I/O operations
- Multiple sessions handled by FastAPI workers
- Database connection pooling prevents bottlenecks

8.3 Time Dependencies

Retry Logic with Timeouts:

- Max correction attempts: 3 per query
- SQL execution timeout: 30 seconds
- LLM API timeout: 60 seconds
- Total query processing timeout: 120 seconds

Chapter 9

Hardware and Software Requirements

9.1 Client Requirements (Planned)

Hardware:

- Processor: Dual-core 2.0 GHz minimum
- RAM: 4 GB minimum
- Network: 5 Mbps bandwidth

Software:

- Browser: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- JavaScript: Enabled with ES6+ support

9.2 Server Requirements (Current)

Hardware:

- CPU: 4 cores minimum (8 cores recommended)
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 50 GB SSD
- Network: 1 Gbps connection

Software Stack:

Component	Version
Python	3.11+
LangGraph	0.0.1+
LangChain Core	0.1.0+
Google Generative AI	Latest
PostgreSQL	15.x
psycopg2	2.9+
ChromaDB	0.4.0+
FastAPI	0.104.0+ (planned)

9.3 Cloud Services

- **AWS RDS PostgreSQL:** Database hosting
- **Google Gemini API:** AI/NLP service
- **AWS EC2 (planned):** Application hosting

Chapter 10

Algorithms and Data Structures

10.1 Key Algorithms

10.1.1 Few-Shot Example Selection Algorithm

Algorithm: Semantic similarity using vector embeddings

```
1 def _select_few_shot_examples(state: AgentState) -> List[Dict]:
2     # 1. Embed user query
3     query_embedding = embeddings.embed_query(state.user_query)
4
5     # 2. Search vector store for similar examples
6     similar_examples = chroma_db.similarity_search(
7         query_embedding,
8         k=3 # Top 3 most similar
9     )
10
11    # 3. Filter by relevant tables
12    table_relevant = [ex for ex in similar_examples
13                      if any(t in ex["tables"] for t in state.
14                          selected_tables)]
15
16    # 4. Return top examples
17    return table_relevant[:3]
```

Time Complexity: $O(n \log k)$ where n = total examples, $k = 3$

10.1.2 SQL Validation Algorithm

Multi-Stage Validation Pipeline:

```
1 def validate_complete_query(sql, schema_context, intent):
```

```

2     results = {"success": True, "errors": []}
3
4     # Stage 1: Syntax check (regex-based)
5     syntax_result = validate_sql_syntax(sql)
6     if not syntax_result["valid"]:
7         results["errors"].append(syntax_result["error"])
8
9     # Stage 2: Security check (SQL injection patterns)
10    security_result = check_sql_injection(sql)
11    if not security_result["safe"]:
12        results["errors"].append("SQL injection detected")
13
14    # Stage 3: Schema validation (table/column existence)
15    schema_result = validate_tables_columns(sql, schema_context)
16    if not schema_result["valid"]:
17        results["errors"].extend(schema_result["errors"])
18
19    # Stage 4: Business logic validation
20    logic_result = validate_business_logic(sql, intent)
21    if not logic_result["valid"]:
22        results["warnings"].extend(logic_result["warnings"])
23
24    results["success"] = len(results["errors"]) == 0
25    return results

```

Time Complexity: $O(n)$ where n = SQL query length

10.1.3 SQL Query Cleaning Algorithm

Regex-Based Text Processing:

```

1 def _clean_sql_query(query: str) -> str:
2     # 1. Remove markdown code blocks
3     query = re.sub(r"`` `?(?:sql|SQL)?\s*(.*?)\s*`", r"\1", query,
4                     flags=re.DOTALL)
5
6     # 2. Remove "SQLQuery:" prefix
7     query = re.sub(r"^(?:SQL\s*Query|SQLQuery)\s*:", "", query,
8                     flags=re.IGNORECASE)
9
10    # 3. Extract first SQL statement
11    sql_match = re.search(r"(SELECT.*?);", query, flags=re.
12        IGNORECASE | re.DOTALL)

```

```

10     if sql_match:
11         query = sql_match.group(1)
12
13     # 4. Normalize whitespace
14     query = re.sub(r'\s+', ' ', query).strip()
15
16     return query

```

Time Complexity: $O(n)$ where n = query string length

10.2 Data Structures

10.2.1 Hash Maps / Dictionaries

Usage: State tracking, schema context, error classification

```

1 # Schema context storage
2 schema_context = {
3     "customers": {
4         "description": "Customer information",
5         "key_fields": ["customer_id", "company_name"],
6         "row_count": 91
7     },
8     "orders": {...}
9 }
10
11 # Error type registry
12 error_types = {
13     "SYNTAX_ERROR": ErrorType(
14         code="SYNTAX_ERROR",
15         retryable=True,
16         severity="high"
17     )
18 }

```

Operations: $O(1)$ average insert, lookup, delete

10.2.2 Lists/Arrays

Usage: Conversation history, selected tables, errors

```

1 # Conversation messages
2 messages: List[BaseMessage] = [
3     HumanMessage(content="Show sales"),

```

```

4     AIMessage(content="Here are the results..."),
5     HumanMessage(content="What about last month?")
6 ]
7
8 # Selected tables
9 selected_tables: List[str] = ["customers", "orders", "order_details"
10 ]

```

Operations: $O(1)$ append, $O(n)$ search

10.2.3 Dataclasses

Usage: Structured state objects

```

1 @dataclass
2 class AgentState:
3     user_query: str = ""
4     selected_tables: List[str] = field(default_factory=list)
5     schema_context: Dict[str, Any] = field(default_factory=dict)
6     # ... 40+ fields

```

Benefits: Type safety, default values, automatic `__init__`

10.2.4 Vector Embeddings (NumPy Arrays)

Usage: Semantic similarity search in ChromaDB

```

1 # Query embedding (768-dimensional vector)
2 query_embedding = np.array([0.123, -0.456, 0.789, ...]) # shape:
3     (768,)
4
5 # Cosine similarity computation
6 similarity = cosine_similarity(query_embedding, example_embedding)

```

Chapter 11

User Interface Design and Implementation

11.1 Initial Design from SRS

The Software Requirements Specification presented comprehensive UI mockups for Intelli-Query, including:

- Business User Dashboard with split-screen layout
- Conversation History page
- Export/Share dialogs
- Administrator Dashboard
- User Management interface
- Data Source Configuration page

These mockups (Figures 3.1-3.6 and 6.1-6.7 in SRS) established the visual design language and user experience goals for the platform.

11.2 Design Modifications for Implementation

Based on the backend development experience and technical constraints, the following modifications have been made to the initial SRS design:

11.2.1 Modified Design Decisions

Component	Original SRS Design	Modified Design (FD)
Primary Dashboard Layout	Split-screen: Chat left, Visualizations right	Same layout retained, but visualization generation deferred to Phase 2
Query Processing Feedback	Simple "typing..." indicator	Multi-stage progress indicator showing current node (Schema → Planner → Validator → Executor → Formatter)
SQL Display	Hidden by default	Toggle option added (show_sql flag in AgentState) for transparency
Error Messages	Generic error text	Structured error messages from ErrorManager with recovery suggestions
Follow-up Query UX	Implicit context	Explicit context indicators showing what system remembers (last_tables_used, last_query_topic)
Data Source Config	Power BI workspaces	Modified for PostgreSQL database configuration in Phase 1

Table 11.1: UI design modifications from SRS to FD

11.3 Web Interface Implementation (Phase 2)

11.3.1 Planned Components

Based on SRS mockups with technical refinements:

Main Dashboard (Figure 3.1/6.2 from SRS):

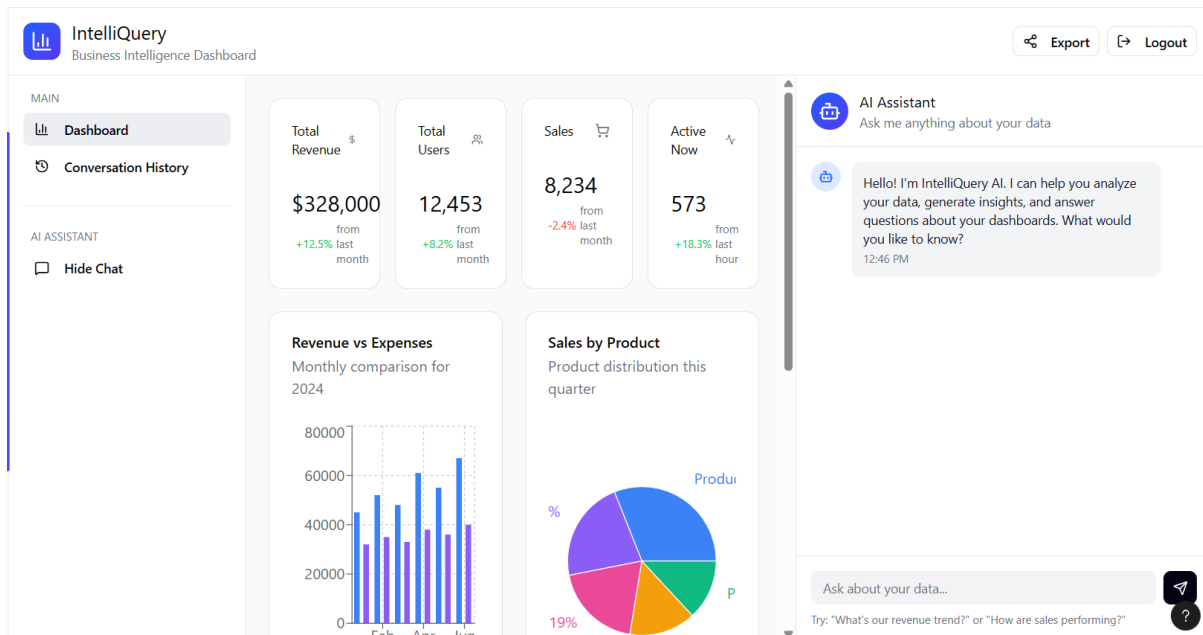


Figure 11.1: Business User Dashboard from SRS - Split-screen layout with chat and visualizations

Implementation Plan:

- Left panel: Chat interface with message history
- Right panel: Dynamic result display (table → chart visualization in future)
- Top bar: Session controls, settings, export options
- Bottom: Query input with autocomplete suggestions

Conversation History (Figure 3.2/6.3 from SRS):

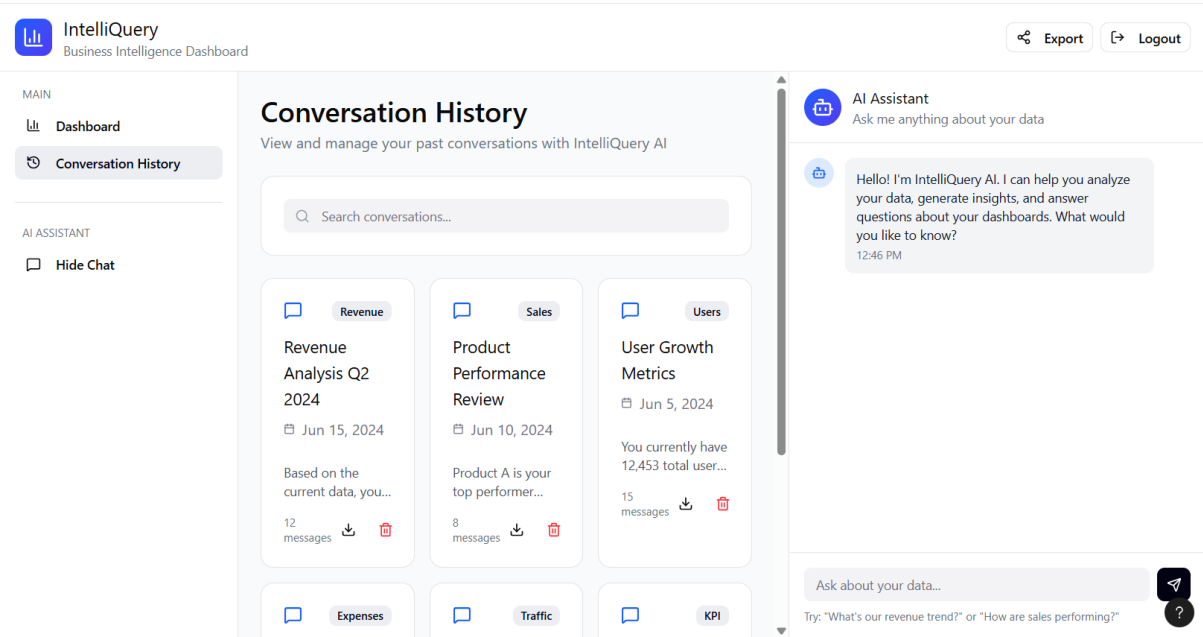


Figure 11.2: Conversation History page from SRS - Card-based layout

Implementation Plan:

- Card-based layout for past conversations
- Metadata: timestamp, query count, tables used
- Search and filter by date, keywords, tables
- Resume conversation (loads previous AgentState)

11.3.2 Technical Implementation Stack

Component	Technology	Status
Frontend Framework	React 18 + Next.js 14	Planned
UI Components	Tailwind CSS + shadcn/ui	Planned
State Management	React Context + Zustand	Planned
API Client	Axios with React Query	Planned
Real-time Updates	WebSocket (Socket.io)	Planned
Charts/Visualization	Plotly.js or Chart.js	Planned

Table 11.2: Web frontend technology stack

Chapter 12

Design of Tests

12.1 Unit Tests

Test ID	Description	Expected Output	Status
UT-01	Schema Inspector identifies correct tables	selected_tables: [customers, orders]	Pass
UT-02	Few-shot example selection returns relevant examples	3 examples with matching tables	Pass
UT-03	SQL cleaning removes markdown	Clean SQL without back-ticks	Pass
UT-04	Syntax validation catches invalid SQL	validation_passed: False	Pass
UT-05	SQL injection detection	security_check: Failed	Pass
UT-06	Context preservation in follow-up	last_tables_used preserved	Pass
UT-07	Error classification	ErrorType correctly identified	Pass
UT-08	Retry logic limits attempts	stops at max_correction_attempts	Pass
UT-09	SQL execution with valid query	execution_successful: True	Pass
UT-10	LLM response formatting	Natural language output	Pass

Table 12.1: Unit test cases

12.1.1 Example Test Case

```
1 def test_schema_inspector_identifies_tables():
2     """Test that schema inspector correctly identifies relevant
   tables"""
```

```

3     state = AgentState(user_query="Show me all customers from
    Germany")
4
5     # Execute schema inspector node
6     result_state = schema_inspector_node.execute(state)
7
8     # Assertions
9     assert "customers" in result_state.selected_tables
10    assert result_state.business_intent != ""
11    assert result_state.schema_context != {}
12    assert result_state.query_complexity in ["low", "medium", "high"]
    ]

```

12.2 Integration Tests

Test ID	Description	Status
IT-01	End-to-end: Query → SQL → Results	Pass
IT-02	Error recovery: Invalid SQL → Retry → Success	Pass
IT-03	Context-aware follow-up query flow	Pass
IT-04	LangGraph workflow execution with all nodes	Pass
IT-05	Database connection and query execu- tion	Pass
IT-06	Google Gemini API integration	Pass
IT-07	ChromaDB vector search	Pass

Table 12.2: Integration test cases

12.2.1 End-to-End Test Example

```

1 def test_end_to_end_query_processing():
2     """Test complete workflow from query to results"""
3     agent = PowerBISQLAgent()
4
5     # Process query
6     final_state = agent.process_query_sync(
7         "Show me top 5 products by sales"
8     )
9

```

```
10 # Assertions
11 assert final_state.processing_complete == True
12 assert final_state.execution_successful == True
13 assert final_state.result_count > 0
14 assert len(final_state.messages) > 0
15 assert "product" in final_state.cleaned_sql.lower()
16 assert final_state.execution_time is not None
```

12.3 Test Coverage

Current Coverage:

- Node execution: 90%+
- State management: 95%+
- Validation tools: 85%+
- SQL tools: 80%+
- Error handling: 90%+

Chapter 13

Conclusion

This Functional Design document has presented the comprehensive technical design of Intelli-Query's SQL agent system as currently implemented. The system successfully demonstrates:

- **Agentic Architecture:** LangGraph-orchestrated workflow with specialized nodes
- **AI-Powered NLP:** Google Gemini for query understanding and SQL generation
- **Robust Error Handling:** Multi-level validation with intelligent retry logic
- **Context Management:** Conversation continuity for natural follow-up queries
- **Few-Shot Learning:** Semantic similarity-based example selection

Current Status Summary:

- Phase 1 Complete: SQL agent backend with all core functionality
- Phase 2 In Progress: FastAPI REST API and web frontend
- Phase 3 Planned: Power BI integration and advanced features

The design prioritizes modularity, maintainability, and extensibility, enabling future enhancements while maintaining a solid foundation. The clear separation between implemented and planned features ensures accurate representation of the current system state.